

week_7

December 8, 2025

1 Week 7 Assignment - Neural Network

Grad 509: Dimensionality Reduction Assignment

1.1 Problem 1

Problem 1: In this problem, you are asked to complete a hand-written digit recognition task for MNIST dataset (i.e. multi-class classification task with 10 classes). In this project, you are asked to design a fully-connected neural network (FCNN).

Hints:

1. The information of MNIST dataset can be found in <https://www.tensorflow.org/datasets/catalog/mnist>;
2. The MNIST dataset can be directly downloaded via:

```
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.load_data()
```

3. Dense layer can be used for realizing the FCNN;
4. Since this problem can be modeled as multi-class classification problem with 10 different classes, the output layer needs to have 10 computing units.

```
[1]: import numpy as np
import tensorflow as tf
from tensorflow.keras import datasets, layers, models

# 1. Load MNIST
(train_images, train_labels), (test_images, test_labels) = datasets.mnist.
    ↪load_data()

print(train_images.shape) # (60000, 28, 28)
print(test_images.shape)  # (10000, 28, 28)
```

```
(60000, 28, 28)
```

```
(10000, 28, 28)
```

```
[3]: # Preprocessing of the data before feeding to the Neural Network

# 2. Normalize to [0, 1]
train_images = train_images.astype("float32") / 255.0
```

```
test_images = test_images.astype("float32") / 255.0

# 3. Flatten: (N, 28, 28) -> (N, 784)
train_images = train_images.reshape(-1, 28 * 28)
test_images = test_images.reshape(-1, 28 * 28)

num_classes = 10
```

[5]: *# The first model consist of classifc fully-connected neural network with 2*
↳hidden layers, and 2 regularization drop-out layers

```
model = models.Sequential([
    layers.Input(shape=(28 * 28,)),          # Input layer (flattened image)
    layers.Dense(256, activation='relu'),    # Hidden layer 1
    layers.Dropout(0.2),                    # Regularization
    layers.Dense(128, activation='relu'),    # Hidden layer 2
    layers.Dropout(0.2),
    layers.Dense(num_classes, activation='softmax') # Output layer
])

# The second model consist of classifc fully-connected neural network with 4
↳hidden layers, to check if by increasing depht we can improve performance

model_2 = models.Sequential([
    layers.Input(shape=(28 * 28,)),          # Input layer (flattened image)
    layers.Dense(256, activation='relu'),    # Hidden layer 1
    layers.Dropout(0.2),                    # Regularization
    layers.Dense(256, activation='relu'),    # Hidden layer 2
    layers.Dropout(0.2),                    # Regularization
    layers.Dense(128, activation='relu'),    # Hidden layer 3
    layers.Dropout(0.2),                    # Regularization
    layers.Dense(128, activation='relu'),    # Hidden layer 4
    layers.Dropout(0.2),
    layers.Dense(num_classes, activation='softmax') # Output layer
])
```

[6]: `model.compile(
 optimizer='adam',
 loss='sparse_categorical_crossentropy',
 metrics=['accuracy']
)`

`model.summary()`

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_8 (Dense)	(None, 256)	200,960
dropout_6 (Dropout)	(None, 256)	0
dense_9 (Dense)	(None, 128)	32,896
dropout_7 (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 10)	1,290

Total params: 235,146 (918.54 KB)

Trainable params: 235,146 (918.54 KB)

Non-trainable params: 0 (0.00 B)

```
[7]: history = model.fit(
    train_images,
    train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=2
)
```

Epoch 1/10

422/422 - 1s - 3ms/step - accuracy: 0.8956 - loss: 0.3540 - val_accuracy: 0.9667
- val_loss: 0.1156

Epoch 2/10

422/422 - 1s - 2ms/step - accuracy: 0.9557 - loss: 0.1471 - val_accuracy: 0.9768
- val_loss: 0.0832

Epoch 3/10

422/422 - 1s - 2ms/step - accuracy: 0.9682 - loss: 0.1052 - val_accuracy: 0.9747
- val_loss: 0.0828

Epoch 4/10

422/422 - 1s - 2ms/step - accuracy: 0.9735 - loss: 0.0837 - val_accuracy: 0.9775
- val_loss: 0.0728

Epoch 5/10

422/422 - 1s - 2ms/step - accuracy: 0.9781 - loss: 0.0699 - val_accuracy: 0.9807
- val_loss: 0.0676

Epoch 6/10

422/422 - 1s - 2ms/step - accuracy: 0.9816 - loss: 0.0582 - val_accuracy: 0.9822
- val_loss: 0.0640

```
Epoch 7/10
422/422 - 1s - 2ms/step - accuracy: 0.9839 - loss: 0.0490 - val_accuracy: 0.9823
- val_loss: 0.0612
Epoch 8/10
422/422 - 1s - 2ms/step - accuracy: 0.9842 - loss: 0.0475 - val_accuracy: 0.9828
- val_loss: 0.0598
Epoch 9/10
422/422 - 1s - 2ms/step - accuracy: 0.9869 - loss: 0.0397 - val_accuracy: 0.9805
- val_loss: 0.0671
Epoch 10/10
422/422 - 1s - 2ms/step - accuracy: 0.9873 - loss: 0.0391 - val_accuracy: 0.9835
- val_loss: 0.0599
```

```
[8]: test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")
```

Test accuracy: 0.9817

```
[9]: pred_probs = model.predict(test_images)
pred_labels = np.argmax(pred_probs, axis=1)

print("First 10 predictions: ", pred_labels[:10])
print("True labels:          ", test_labels[:10])
```

```
313/313          0s 422us/step
First 10 predictions: [7 2 1 0 4 1 4 9 5 9]
True labels:         [7 2 1 0 4 1 4 9 5 9]
```

```
[10]: model_2.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model_2.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 256)	200,960
dropout_8 (Dropout)	(None, 256)	0
dense_12 (Dense)	(None, 256)	65,792
dropout_9 (Dropout)	(None, 256)	0

dense_13 (Dense)	(None, 128)	32,896
dropout_10 (Dropout)	(None, 128)	0
dense_14 (Dense)	(None, 128)	16,512
dropout_11 (Dropout)	(None, 128)	0
dense_15 (Dense)	(None, 10)	1,290

Total params: 317,450 (1.21 MB)

Trainable params: 317,450 (1.21 MB)

Non-trainable params: 0 (0.00 B)

```
[11]: history = model_2.fit(
    train_images,
    train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=2
)
```

Epoch 1/10

422/422 - 2s - 4ms/step - accuracy: 0.8720 - loss: 0.4085 - val_accuracy: 0.9662
- val_loss: 0.1123

Epoch 2/10

422/422 - 1s - 3ms/step - accuracy: 0.9543 - loss: 0.1581 - val_accuracy: 0.9747
- val_loss: 0.0859

Epoch 3/10

422/422 - 1s - 3ms/step - accuracy: 0.9664 - loss: 0.1167 - val_accuracy: 0.9783
- val_loss: 0.0761

Epoch 4/10

422/422 - 1s - 3ms/step - accuracy: 0.9714 - loss: 0.0978 - val_accuracy: 0.9787
- val_loss: 0.0674

Epoch 5/10

422/422 - 1s - 3ms/step - accuracy: 0.9755 - loss: 0.0828 - val_accuracy: 0.9812
- val_loss: 0.0634

Epoch 6/10

422/422 - 1s - 3ms/step - accuracy: 0.9780 - loss: 0.0726 - val_accuracy: 0.9810
- val_loss: 0.0652

Epoch 7/10

422/422 - 1s - 3ms/step - accuracy: 0.9817 - loss: 0.0627 - val_accuracy: 0.9812

```
- val_loss: 0.0670
Epoch 8/10
422/422 - 1s - 3ms/step - accuracy: 0.9820 - loss: 0.0589 - val_accuracy: 0.9817
- val_loss: 0.0595
Epoch 9/10
422/422 - 1s - 3ms/step - accuracy: 0.9831 - loss: 0.0547 - val_accuracy: 0.9807
- val_loss: 0.0657
Epoch 10/10
422/422 - 1s - 3ms/step - accuracy: 0.9857 - loss: 0.0476 - val_accuracy: 0.9817
- val_loss: 0.0639
```

```
[12]: test_loss, test_acc = model_2.evaluate(test_images, test_labels, verbose=0)
      print(f"Test accuracy: {test_acc:.4f}")
```

Test accuracy: 0.9814

```
[13]: pred_probs = model_2.predict(test_images)
      pred_labels = np.argmax(pred_probs, axis=1)

      print("First 10 predictions: ", pred_labels[:10])
      print("True labels:          ", test_labels[:10])
```

```
313/313          0s 538us/step
First 10 predictions: [7 2 1 0 4 1 4 9 5 9]
True labels:         [7 2 1 0 4 1 4 9 5 9]
```

As we see, by increasing more layers, we didnt improve the performance, which actually was quite good with the first iteration. Also may be a more sophisticate arquitecure could improve performance, but we will check on that on the next problem.

1.2 Problem 2

Problem 2: In this problem, you will focus on the same hand-written digit recognition task for MNIST dataset. In this project, you are asked to design a convolutional neural network (CNN) with two convolutional layers and two max-pooling layers.

Hints: 1. Realization of convolutional layer can be found in: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D; 2. Realization of max-pooling layer can be found in: https://www.tensorflow.org/api_docs/python/tf/compat/v1/layers/MaxPooling2D; 3. Realization of flatten layer can be found in: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Flatten;

```
[17]: # 1. Load data
      (train_images, train_labels), (test_images, test_labels) = datasets.mnist.
      ↪load_data()

      # 2. Normalize to [0, 1]
      train_images = train_images.astype("float32") / 255.0
      test_images  = test_images.astype("float32") / 255.0
```

```

# 3. Add channel dimension: (N, 28, 28) -> (N, 28, 28, 1)
train_images = np.expand_dims(train_images, axis=-1)
test_images = np.expand_dims(test_images, axis=-1)

num_classes = 10

```

```

[18]: model = models.Sequential([
    # First conv block
    layers.Conv2D(filters=32,
                  kernel_size=(3, 3),
                  activation='relu',
                  input_shape=(28, 28, 1)),
    layers.MaxPooling2D(pool_size=(2, 2)),

    # Second conv block
    layers.Conv2D(filters=64,
                  kernel_size=(3, 3),
                  activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),

    # Flatten + fully connected layers
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(num_classes, activation='softmax') # 10 classes (0-9)
])

```

```

[19]: model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0

flatten_1 (Flatten)	(None, 1600)	0
dense_18 (Dense)	(None, 64)	102,464
dense_19 (Dense)	(None, 10)	650

Total params: 121,930 (476.29 KB)

Trainable params: 121,930 (476.29 KB)

Non-trainable params: 0 (0.00 B)

```
[20]: history = model.fit(
    train_images,
    train_labels,
    epochs=10,
    batch_size=128,
    validation_split=0.1,
    verbose=2
)
```

Epoch 1/10

422/422 - 8s - 18ms/step - accuracy: 0.9193 - loss: 0.2779 - val_accuracy: 0.9797 - val_loss: 0.0768

Epoch 2/10

422/422 - 6s - 15ms/step - accuracy: 0.9788 - loss: 0.0695 - val_accuracy: 0.9853 - val_loss: 0.0537

Epoch 3/10

422/422 - 6s - 15ms/step - accuracy: 0.9854 - loss: 0.0467 - val_accuracy: 0.9868 - val_loss: 0.0480

Epoch 4/10

422/422 - 6s - 15ms/step - accuracy: 0.9887 - loss: 0.0358 - val_accuracy: 0.9890 - val_loss: 0.0407

Epoch 5/10

422/422 - 7s - 16ms/step - accuracy: 0.9916 - loss: 0.0281 - val_accuracy: 0.9907 - val_loss: 0.0368

Epoch 6/10

422/422 - 7s - 16ms/step - accuracy: 0.9929 - loss: 0.0228 - val_accuracy: 0.9917 - val_loss: 0.0302

Epoch 7/10

422/422 - 6s - 15ms/step - accuracy: 0.9938 - loss: 0.0192 - val_accuracy: 0.9908 - val_loss: 0.0328

Epoch 8/10

422/422 - 7s - 15ms/step - accuracy: 0.9950 - loss: 0.0157 - val_accuracy: 0.9922 - val_loss: 0.0305

```
Epoch 9/10
422/422 - 7s - 15ms/step - accuracy: 0.9959 - loss: 0.0127 - val_accuracy:
0.9915 - val_loss: 0.0327
Epoch 10/10
422/422 - 7s - 16ms/step - accuracy: 0.9959 - loss: 0.0122 - val_accuracy:
0.9927 - val_loss: 0.0317
```

```
[21]: test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=0)
      print(f"Test accuracy: {test_acc:.4f}")
```

Test accuracy: 0.9916

1.3 Problem 3

Problem 3: The IMDB dataset is a set of 50,000 highly polarized reviews from the Internet Movie Database. In this problem, your task is to develop an LSTM-based recurrent neural network model to automatically identify whether a given review is positive or negative.

Hints: 1. The information of the IMDB dataset can be found in https://www.tensorflow.org/api_docs/python/tf/keras/datasets/imdb.

By looking at the website, we see that the dataset is already tokenized, so we only need to set a couple of hyperparameters

```
[6]: import tensorflow as tf
      from tensorflow.keras import datasets, layers, models
      from tensorflow.keras.preprocessing.sequence import pad_sequences

      # Hyperparameters
      num_words = 10000    # Only top 10k most frequent words
      maxlen = 200        # maximum length of each review

      # 2. Cargar datos
      (x_train, y_train), (x_test, y_test) = datasets.imdb.
      ↪load_data(num_words=num_words)

      print(len(x_train), len(x_test))    # 25000, 25000
      print(x_train[0][:10])
```

25000 25000

[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65]

Fill each sequence, so they all have the maximum length

```
[7]: x_train_padded = pad_sequences(x_train, maxlen=maxlen)
      x_test_padded = pad_sequences(x_test, maxlen=maxlen)

      print(x_train_padded.shape)    # (25000, 200)
      print(x_test_padded.shape)    # (25000, 200)
```

```
(25000, 200)
(25000, 200)
```

```
[9]: embedding_dim = 64

model = models.Sequential([
    layers.Embedding(input_dim=num_words,
                     output_dim=embedding_dim),
    layers.LSTM(64),
    layers.Dense(1, activation='sigmoid')    # binary target
])
```

```
[10]: model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_2 (LSTM)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

```
[11]: history = model.fit(
        x_train_padded,
        y_train,
        epochs=5,
        batch_size=128,
        validation_split=0.2,
        verbose=2
    )
```

```
)
```

Epoch 1/5

157/157 - 20s - 126ms/step - accuracy: 0.7725 - loss: 0.4572 - val_accuracy: 0.8744 - val_loss: 0.3081

Epoch 2/5

157/157 - 18s - 117ms/step - accuracy: 0.9068 - loss: 0.2440 - val_accuracy: 0.8682 - val_loss: 0.3074

Epoch 3/5

157/157 - 17s - 107ms/step - accuracy: 0.9326 - loss: 0.1824 - val_accuracy: 0.8612 - val_loss: 0.4008

Epoch 4/5

157/157 - 18s - 114ms/step - accuracy: 0.9520 - loss: 0.1370 - val_accuracy: 0.8698 - val_loss: 0.3537

Epoch 5/5

157/157 - 18s - 116ms/step - accuracy: 0.9668 - loss: 0.0970 - val_accuracy: 0.8648 - val_loss: 0.3834

```
[12]: test_loss, test_acc = model.evaluate(x_test_padded, y_test, verbose=0)
      print(f"Test accuracy: {test_acc:.4f}")
```

Test accuracy: 0.8596